Generación de Código Intermedio

Generación de Código Intermedio

- **Introducción.**
- **Lenguajes Intermedios.** Código de tres direcciones. Tipos de sentencias. Traducción a código intermedio. Posibles implementaciones.
- **Declaraciones.** Manejo de información de alcance.
- **Sentencias de Asignación.**
- **Expresiones Booleanas.**
- **Sentencias *Case.* Llamadas a Procedimiento**

Generación de Código Intermedio

- El código intermedio es código *independiente* de la máquina destino, pero conceptualmente *cercano* a ella.

- El programa en lenguaje fuente es traducido a un programa equivalente en código intermedio por el generador de código intermedio.

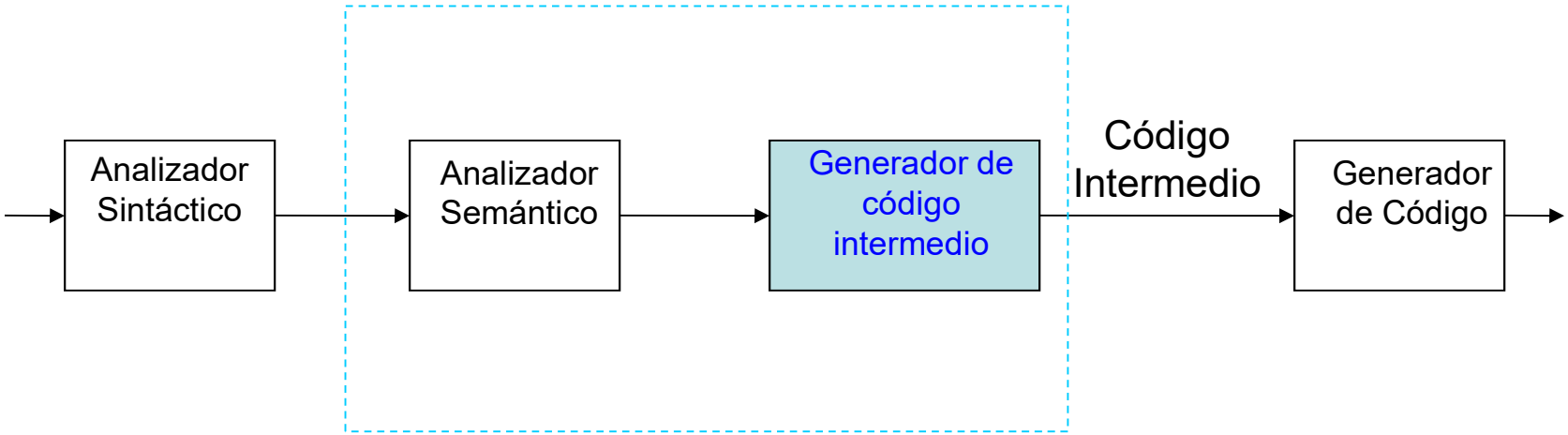
Generación de Código Intermedio

Beneficios

- El análisis se independiza del lenguaje destino.
- Es posible aplicar optimizaciones independientes del lenguaje destino.
- Portabilidad. Un compilador para una máquina destino diferente requiere modificar solo algunos componentes.

Generación de Código Intermedio

Ubicación



Generación de Código Intermedio

- **Introducción.**
- **Lenguajes Intermedios.** Código de tres direcciones. Tipos de sentencias. Traducción a código intermedio. Posibles implementaciones.
- **Declaraciones.** Manejo de información de alcance.
- **Sentencias de Asignación.**
- **Expresiones Booleanas.**
- **Sentencias *Case.* Llamadas a Procedimiento**

Lenguajes Intermedios

Existen diferentes tipos de lenguajes intermedios

- Árboles sintácticos
- Notación postfija
- Código de tres-direcciones
  - Lo emplearemos en DDS y ET para discutir la generación de código intermedio
- Algunos lenguajes de programación tienen sus propios lenguajes intermedios:
  - Java – Java virtual machine
  - .NET – MSIL
  - Prolog – Warren abstract machine
  - Pascal – Máquina de Ejecución para Pascal (MEPA)

Código de Tres Direcciones

Las sentencias de este lenguaje son de la forma:

x := y *op* z

donde *x, y, z* son nombres, constantes o variables temporales generadas por el compilador, y *op* es un operador aritmético o lógico.

Una sentencia (en lenguaje fuente) como:

d := a+b*c

Podría ser traducida a código de 3D como:

t_1 := b * c
d := a + t_1

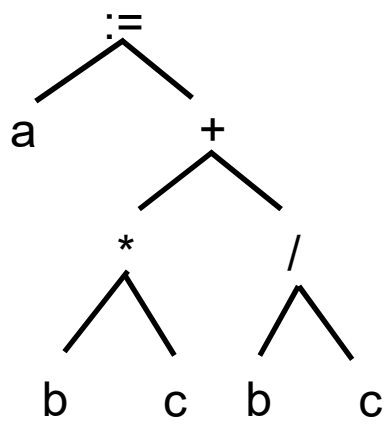
donde t_1 es una variable temporal generada por el compilador.

Lenguajes Intermedios. Ejemplos

sentencia: a := b * c + b / c

n. postfija: abc*bc/+ :=

a. sintáctico:



código de tres-direcciones:

t_1 := b * c
t_2 := b / c
t_3 := t_1 + t_2
a := t_3

Código de Tres Direcciones

- Las sentencias de tres direcciones pueden contener *nombres* (a, b, c, etc.). En la implementación, cada nombre puede ser reemplazado por un puntero a su entrada en la TS, donde se almacena la información relacionada.

- Las sentencias pueden tener etiquetas simbólicas y existen sentencias para manejar el flujo de control.

- Una etiqueta simbólica representa el índice de una sentencia en el arreglo o lista que contiene al código intermedio.

- Los índices reales pueden sustituir a los labels ya sea mediante un paso separado o usando "*back-patching*".

Código de Tres Direcciones

Tipos de sentencias más comunes

- Sentencias de asignación de la forma x := y *op* z, donde *op* es un operador binario aritmético o lógico.
- Sentencias de asignación de la forma x := *op* y, donde *op* es un operador unario.
- Sentencias de copia de la forma x := y donde el valor de y es asignado a x.
- La sentencia de salto incondicional goto L. La sentencia con label L es la próxima a ser ejecutada.
- Sentencias de salto condicional como if x *relop* y goto L, donde "*relop*" es un operador relacional (<, =, <=, etc.)

Código de Tres Direcciones

Tipos de sentencias más comunes

- Sentencias de la forma "param x" y "call p,n" para llamadas a procedimientos y "return y", donde y representa el valor retornado. Su uso típico es como una secuencia de sentencias:

  param x_1
  param x_2
  …
  param x_n
  call p, n

  generadas como parte de la llamada al procedimiento $p(x_1,x_2,\ldots,x_n)$. El entero n indicando el número de parámetros actuales en "call p, n" no es redundante porque las llamadas pueden anidarse.

Código de Tres Direcciones

- Sentencias indexadas de la forma x := y[j] y x[j] := y.

  La primera, asigna a x el valor en la posición j unidades de memoria después de la posición y.

  La sentencia x[j] := y setea el contenido de la posición j unidades después de x con el valor de y. En ambos casos, x, y, j se refieren a objetos de datos.

- Asignaciones de direcciones y punteros de la forma x := &y,
  x := *y, y *x := y.

Código de Tres Direcciones

- El conjunto de instrucciones debe ser lo suficientemente rico como para implementar las operaciones del lenguaje fuente.
- Un conjunto de instrucciones restringido puede forzar al front-end a generar largas secuencias de sentencias para ciertas operaciones del lenguaje fuente.
- En tal caso, el optimizador y el generador de código tendrían más trabajo..

Traducción Dirigida por la Sintaxis a Código Intermedio

Producción	Reglas Semánticas
S → **id** := E	S.code := E.code || gen(**id**.place ':=' E.place)
E → E_1 + E_2	E.place := newtemp; E.code := E_1.code || E_2.code || gen(E.place ':=' E_1.place '+' E_2.place)
E → E_1 * E_2	E.place := newtemp; E.code := E_1.code || E_2.code || gen(E.place ':=' E_1.place '*' E_2.place)
E → - E_1	E.place := newtemp; E.code := E_1.code || gen(E.place ':=' 'uminus' E_1.place)
E → (E_1)	E.place := E_1.place; E.code := E_1.code
E → **id**	E.place := **id**.place; E.code := ' '

Traducción Dirigida por la Sintaxis a Código Intermedio

- El atributo sintetizado *S.code* contiene el código de tres direcciones para la sentencia de asignación generada desde *S*.

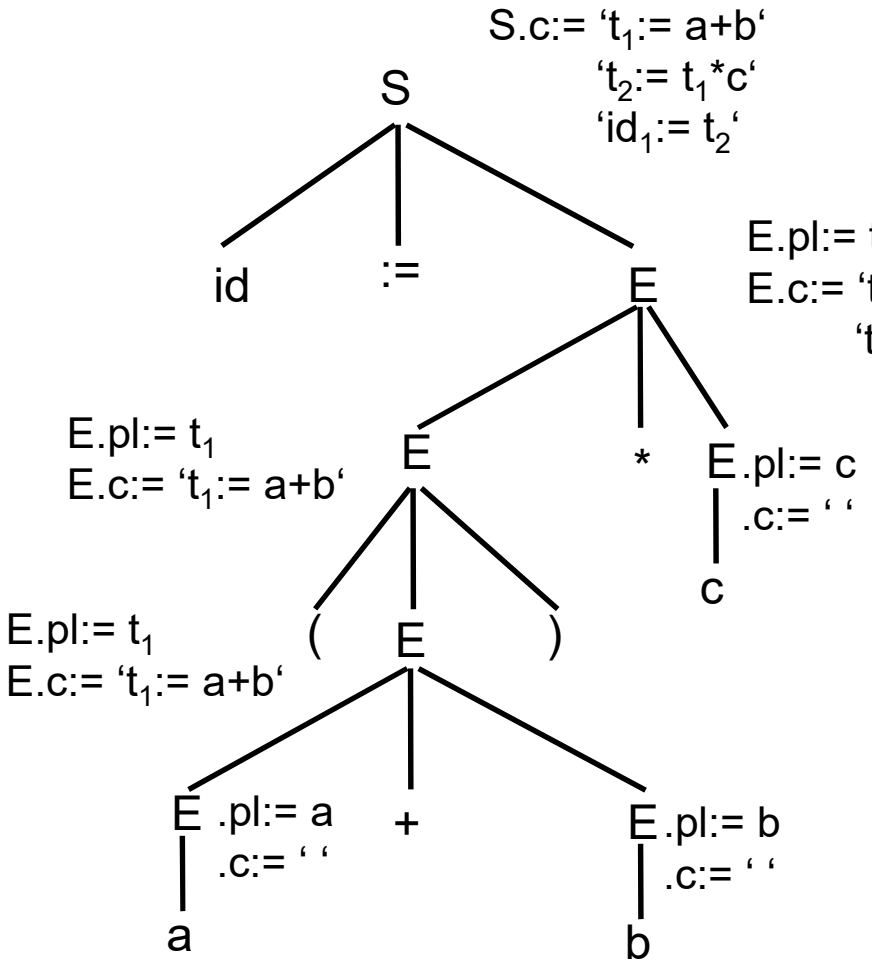
- El no terminal E tiene dos atributos:
  - *E.place*, que es el nombre que contendrá el valor de E y
  - *E.code*, la secuencia de sentencias de tres direcciones que es la traducción de lo generado por E.

- La función *newtemp* retorna, en cada llamada, un nuevo temporal t_1, t_2,… .

Traducción Dirigida por la Sintaxis a Código Intermedio

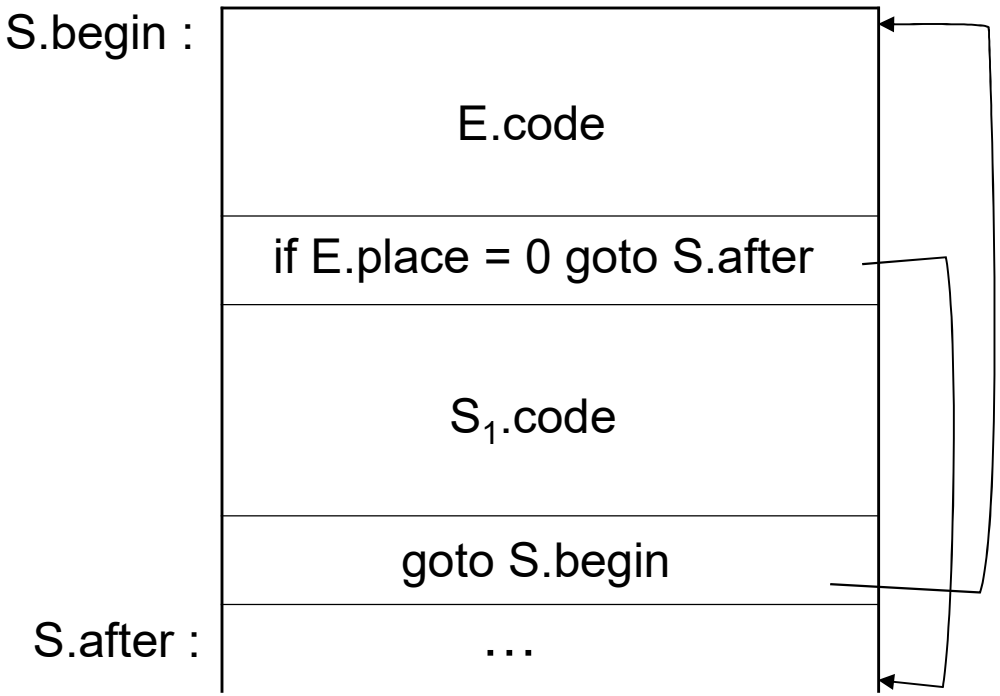
Producción	Reglas Semánticas
S → **id** := E	S.code := E.code || gen(**id**.place ':=' E.place)
E → E_1 + E_2	E.place := newtemp; E.code := E_1.code || E_2.code || gen(E.place ':=' E_1.place '+' E_2.place)
E → E_1 * E_2	E.place := newtemp; E.code := E_1.code || E_2.code || gen(E.place ':=' E_1.place '*' E_2.place)
E → - E_1	E.place := newtemp; E.code := E_1.code || gen(E.place ':=' 'uminus' E_1.place)
E → (E_1)	E.place := E_1.place; E.code := E_1.code
E → **id**	E.place := **id**.place; E.code := ' '

Traducción Dirigida por la Sintaxis



id_1:= (a+b) * c

Producción	Reglas Semánticas
S → **id** := E	S.code := E.code ‖ gen(**id**.place ‘:=‘ E.place)
E → E_1 + E_2	E.place := newtemp; E.code := E_1.code ‖ E_2.code ‖ gen(E.place ‘:=‘ E_1.place ‘+’ E_2.place)
E → E_1 * E_2	E.place := newtemp; E.code := E_1.code ‖ E_2.code ‖ gen(E.place ‘:=‘ E_1.place ‘*’ E_2.place)
E → (E_1)	E.place := E_1.place; E.code := E_1.code
E → **id**	E.place := **id**.place; E.code := ‘ ‘



Producción	Reglas Semánticas
S → **while** E **do** S_1	S.begin := newlabel; S.after := newlabel; S.code := gen(S.begin ‘:’) || E.code || gen(‘if’ E.place ‘=‘ ‘0’ ‘goto’ S.after) || S_1.code || gen(‘goto’ S.begin) || gen(S.after ‘:’)

Implementación de Sentencias de Tres Direcciones

- Las sentencias de tres direcciones pueden ser implementadas mediante registros con campos para el operador y para los operandos.

- Tres representaciones son las más importantes:
  - cuadruplas
  - ternas
  - ternas indirectas

Implementación de Sentencias de Tres Direcciones

Cuadruplas

- Consisten en registros de cuatro campos: *op*, *arg*1, *arg2* y *result*.
- El campo *op* contiene un código interno para el operador.
- Las sentencias con operadores unarios como x:= -y no usan *arg2*. Los operadores como param no usan *arg2* ni *result*.
- Las sentencias condicionales colocan el label destino en *result*.
- Los contenidos de los campos *arg1*, *arg2* y *result* son normalmente punteros a las entradas en la TS para los nombres representados por estos campos. Si es así, **los nombres temporales deben ser ingresados en la TS cuando son creados**.

Implementación de Sentencias de Tres Direcciones

Cuadruplas

Ejemplo: a:= b*-c + b * -c

	op	arg1	arg2	result
(0)	uminus	c		t_1
(1)	*	b	t_1	t_2
(2)	uminus	c		t_3
(3)	*	b	t_3	t_4
(4)	+	t_2	t_4	t_5
(5)	:=	t_5		a

Implementación de Sentencias de Tres Direcciones

Ternas

- Para evitar ingresar los temporales en la TS, podemos referirnos a ellos por la posición de la sentencia que los computa.
- Si es así, las sentencias de tres direcciones pueden representarse mediante registros de tres campos: *op*, *arg*1 y *arg*2.
- Los campos *arg1* y *arg2* son punteros a la TS o punteros a la estructura de ternas (para los temporales).

Implementación de Sentencias de Tres Direcciones

Ternas

Ejemplo: a:= b*-c + b * -c

	op	arg1	arg2
(0)	uminus	c	
(1)	*	b	**(0)**
(2)	uminus	c	
(3)	*	b	**(2)**
(4)	+	(1)	**(3)**
(5)	assign	a	**(4)**

Implementación de Sentencias de Tres Direcciones

Ternas Indirectas

- Se agrega una lista de punteros a ternas, además de la lista de ternas.
- Se agrega un nivel de *indirección*.

Implementación de Sentencias de Tres Direcciones

Ternas Indirectas

Ejemplo: a:= b*-c + b * -c

	sentencia
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

	op	arg1	arg2
(14)	uminus	c	
(15)	*	b	(14)
(16)	uminus	c	
(17)	*	b	(16)
(18)	+	(15)	(17)
(19)	assign	a	(18)

Generación de Código Intermedio

- **Introducción.**
- **Lenguajes Intermedios.** Código de tres direcciones. Tipos de sentencias. Traducción a código intermedio. Posibles implementaciones.
- **Declaraciones.** Manejo de información de alcance.
- **Sentencias de Asignación.**
- **Expresiones Booleanas.**
- **Sentencias *Case.* Llamadas a Procedimiento**

Declaraciones

- A medida que examinamos la secuencia de declaraciones en un procedimiento o bloque, podemos ir reservando espacio de almacenamiento para los nombres encontrados.

- Para cada nombre local, podemos crear una entrada en la TS con información como el tipo y la dirección relativa para el nombre.

- La dirección relativa consiste en un desplazamiento a partir de la dirección base del área de datos estática o del campo para datos locales en un registro de activación.

Declaraciones

Declaraciones en Procedimientos

- La sintaxis de lenguajes como C o Pascal permite que todas las declaraciones de un procedimiento sean procesadas como un grupo.

- Podemos considerar el siguiente esquema de traducción para procesar declaraciones.

Declaraciones

P →	{ offset := 0} D
D → D ; D	
D → **id** : T	{ enter(**id**.name,T.type,offset);
	offset := offset + T.width }
T → **integer**	{ T.type := integer ;
	T.width := 4 }
T → **real**	{ T.type := real ;
	T.width := 4 }
T → **array**[**num**] **of** T_1	{ T.type := array(**num**.val, T_1.type);
	T.width := **num**.val x T_1.width }
T → ^T_1	{ T.type := pointer(T_1.type) ;
	T.width := 4 }

Declaraciones

Declaraciones en Procedimientos

- A medida que analizamos cada nombre, lo ingresamos en la TS con su tipo y con desplazamiento igual al valor actual de offset. Luego, *offset* es incrementado de acuerdo al tamaño del objeto de dato.

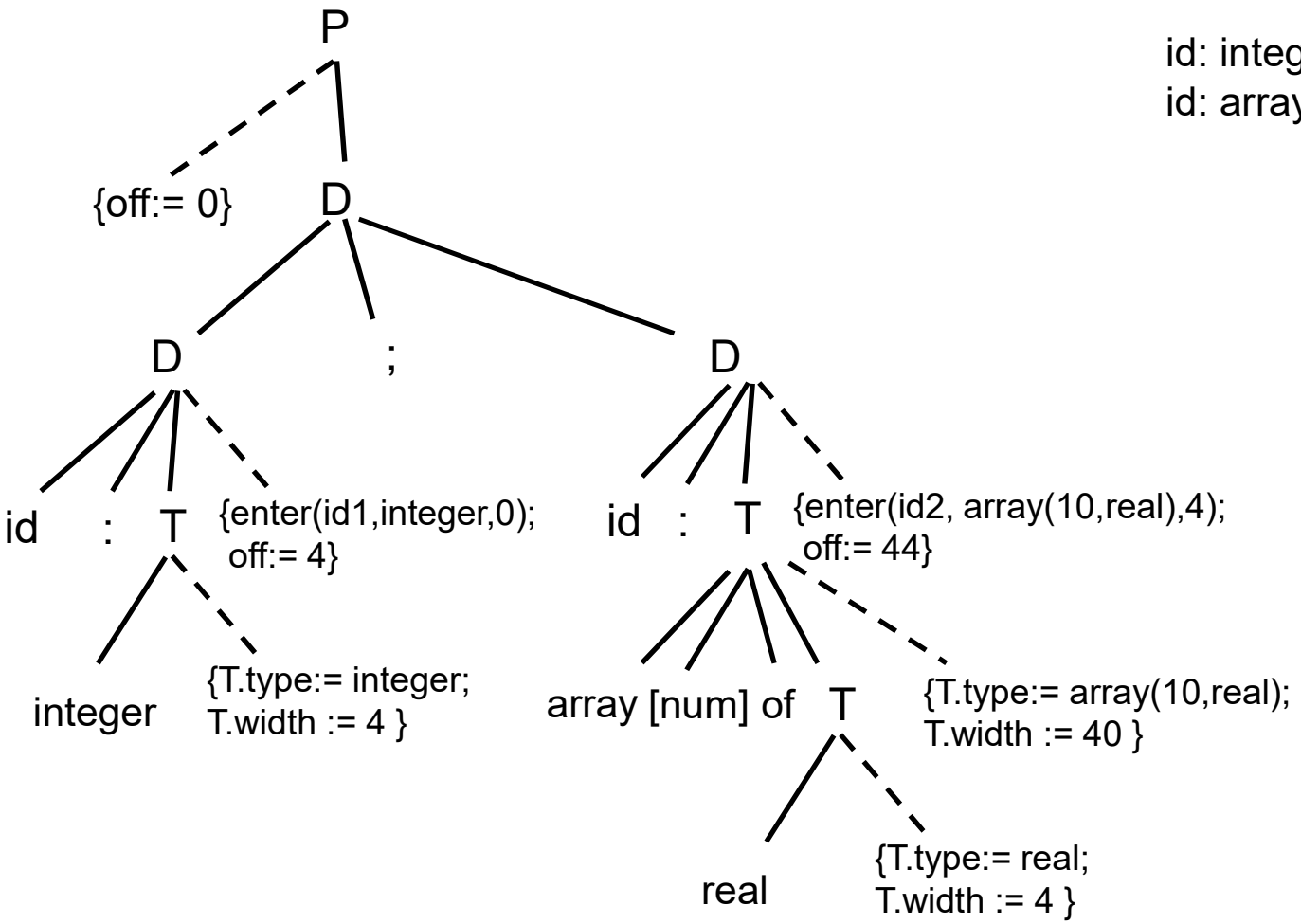
- El procedimiento *enter*(*name*,*type*,*offset*) crea una entrada en la TS para *name* dado su tipo *type* y dirección relativa *offset*.

- Usamos los atributos sintetizados type y width del no terminal T para indicar el tipo y el tamaño o número de unidades de memoria necesarias para los objetos de ese tipo.

Declaraciones

P →	{ offset := 0} D
D → D ; D	
D → **id** : T	{ enter(**id**.name,T.type,offset);
	offset := offset + T.width }
T → **integer**	{ T.type := integer ;
	T.width := 4 }
T → **real**	{ T.type := real ;
	T.width := 4 }
T → **array**[**num**] **of** T_1	{ T.type := array(**num**.val, T_1.type);
	T.width := **num**.val x T_1.width }
T → ^T_1	{ T.type := pointer(T_1.type) ;
	T.width := 4 }

Declaraciones - Ejemplo



id: integer;
id: array [10] of real

Declaraciones

Declaraciones en Procedimientos Anidados

- Creamos una Tabla de Símbolos para cada procedimiento. Consideramos las siguientes operaciones:

  mktable(previous) – crea una nueva TS donde *previous* es la TS padre de esta nueva TS.

  enter(TS,name,type,offset) – crea una nueva entrada para *name* en la TS dada.

  enterproc(TS,name,newTS) – crea una nueva entrada para el nombre del procedimiento en la TS de su padre.

  addwidth(TS,width) – almacena el tamaño total de todas las entradas de la TS en el *header* de la tabla.

- Empleamos dos pilas:

  tblptr – para punteros a las tablas de símbolos.

  offset – para los offsets actuales de las tablas en tblptr

Declaraciones en Procedimientos Anidados

P → M D { addwidth(top(tblptr),top(offset)); pop(tblptr); pop(offset) }

M → λ { t:= mktable(nil); push(t,tblptr); push(0,offset) }

D → D ; D

D → proc **id** N D ; S
{ t:= top(tblptr); addwidth(t,top(offset));
pop(tblptr); pop(offset);
enterproc(top(tblptr),id.name,t) }

D → **id** : T { enter(top(tblptr),id.name,T.type,top(offset));
top(offset):= top(offset)+T.width }

N → λ { t:= mktable(top(tblptr)); push(t,tblptr); push(0,offset) }

Declaraciones en Procedimientos. Ejemplo

```
1  id: integer;  2
   proc id_p  3
       id: array [10] of real;  4
       …
       s
   /* end proc */  5
   id: real  6
7
```

TS **t**

nil		

TblPtr: ^t — Offset: 0

nil		
id1	integer	0

TblPtr: ^t — Offset: 4

TS **t1**

t		
id2	array(10,real)	0

nil		
id1	integer	0

TblPtr: ^t, ^t1 — Offset: 4, 40

t	40	
id2	array(10,real)	0

nil		
id1	integer	0
id_p		t1

TblPtr: ^t — Offset: 4

Declaraciones en Procedimientos. Ejemplo

```
1  id: integer;                2
   proc id_p                   3
       id: array [10] of real; 4
       …
       s                       5
   /* end proc */
   id: real                    6
7
```

TS **t1**

t	40	
id2	array(10,real)	0

TS **t**

nil		
id1	integer	0
id_p		t1

TblPtr: ^t

Offset: 4

t	40	
id2	array(10,real)	0

nil	8	
id1	integer	0
id_p		t1
id3	real	4

TblPtr: ^t

Offset: 8

Nombres de Campos en Registros

En el siguiente esquema de traducción se enfatiza la similitud en el procesamiento de nombres de campos de registros y de declaraciones en procedimientos anidados.

T → **record** L D **end** { T.type:= record(top(tblptr));
T.width:= top(offset);
pop(tblptr); pop(offset) }

L → λ { t:= mktable(nil);
push(t,tblptr);
push(0,offset) }

Nombres de Campos en Registros

T → **record** L D **end** { T.type:= record(top(tblptr));
T.width:= top(offset);
pop(tblptr); pop(offset) }

L → λ { t:= mktable(nil);
push(t,tblptr);
push(0,offset) }

- Luego de leer **record**, se crea una nueva TS para los nombres de los campos.
- Un puntero a ella es apilado en **tblptr** y la dirección relativa 0 es apilada en **offset**.
- Por lo tanto, la acción asociada a *D* → ***id*** *: T* inserta la información para el nombre de campo **id** en la TS del registro.
- Las acciones siguientes a **end**, retornan el tamaño de todos los campos del registro (contenido al tope de la pila **offset**).
- El puntero a la TS del registro (presente en el valor de T.Type) puede ser usado por el generador de código destino para recuperar tipos y tamaños de los campos del registro.

Generación de Código Intermedio

- **Introducción.**
- **Lenguajes Intermedios.** Código de tres direcciones. Tipos de sentencias. Traducción a código intermedio. Posibles implementaciones.
- **Declaraciones.** Manejo de información de alcance.
- **Sentencias de Asignación.**
- **Expresiones Booleanas.**
- **Sentencias *Case.* Llamadas a Procedimiento**

Código Intermedio para Asignaciones

Podríamos considerar el siguiente ET para traducir asignaciones:

S → **id** := E	{ p := **lookup(id.name)** ; **if** p <> nil **then** emit(p ':=' E.place) **else** error }
E → E_1 + E_2	{ E.place := newtemp ; emit(E.place ':=' E_1.place '+' E_2.place) }
E → E_1 * E_2	{ E.place := newtemp ; emit(E.place ':=' E_1.place '*' E_2.place) }
E → - E_1	{ E.place := newtemp ; emit(E.place ':=' 'uminus' E_1.place) }
E → (E_1)	{ E.place := E_1.place }
E → **id**	{ p := **lookup(id.name)** ; **if** p <> nil **then** E.place := p **else** error }

Código Intermedio para Asignaciones

Teniendo en cuenta lo expuesto al considerar las declaraciones en procedimientos anidados la función **lookup(id.name)** debería:

- Chequear si "name" aparece en la TS actual, accesible mediante top(tblptr).
- En caso contrario, lookup usa el puntero en el header de esa tabla para acceder a la TS del procedimiento contenedor y busca en esa TS.
- Así sucesivamente, hasta alcanzar (eventualmente) el alcance global o el predefinido.
- Si el nombre no es encontrado en ninguno de estos alcances, lookup retorna **nil**.

Conversión de Tipos

- Hemos visto como computar expresiones de tipos, en forma dirigida por la sintaxis, empleando más de un tipo de dato.

- También es tarea del compilador, generar las sentencias de tres direcciones necesarias para realizar cualquier conversión de tipos automática que sea requerida.

- Asumiremos que disponemos de dos tipos básicos, *integer* y *real*, y que convertimos enteros en reales.

- También asumimos que existe una función inttoreal y dos operadores "+" diferentes: int+ y real+.

Reglas Semánticas para E_1 -> E_2 + E_3

E_1.place := newtemp();
if E_2.type = integer **and** E_3.type = integer **then**
begin
    emit(E_1.place ":=" E_2.place "int+" E_3.place);
    E_1.type := integer
end
else if E_2.type = real **and** E_3.type = real **then**
begin
    emit(E_1.place ":=" E_2.place "real+" E_3.place);
    E_1.type = real
end
else if E_2.type = integer **and** E_3.type = real **then**
begin
    u := newtemp();
    emit(u ":= inttoreal" E_2.place);
    emit(E_1.place ":=" u "real+" E_3.place);
    E_1.type = real
end
else if

Reglas Semánticas para E_1 -> E_2 + E_3

...

else if E_2.type = real **and** E_3.type = integer **then**
begin
    u := newtemp();
    emit(u ":= inttoreal" E_2.place);
    emit(E_1.place ":=" E_2.place "real+" u);
    E_1.type := real
end
else E_1.type = type_error;

También, serían necesarias reglas semánticas similares para

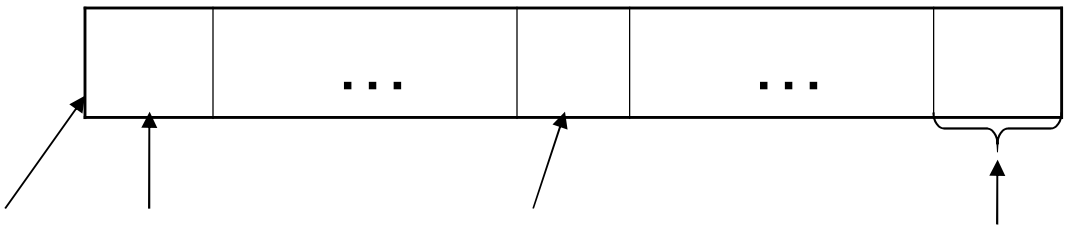
E_1 -> E_2 * E_3

Usando los operadores "int*" y "real*".

Accediendo a Elementos de Arreglos

- Los elementos de arreglos pueden ser accedidos rápidamente si los elementos son almacenados en un bloque posiciones consecutivas.

Array unidimensional **A**:



base$_A$ es la dirección de la primera posición del arreglo A,

width es el tamaño de cada elemento.

low es el índice del primer elemento

Posición de A[i] sería $base_A + (i - low) * width$

Accediendo a Elementos de Arreglos

$base_A$+(i-low)*width

Es equivalente a: i*width + ($base_A$-low*width)



debería ser computado en ejecución

puede ser computado en compilación

- Entonces, la posición de A[i] puede ser computada en ejecución evaluando la fórmula *i*width + c,* donde *c* es *$base_A$-low*width* que ya es evaluado en compilación (y puede residir en la TS).

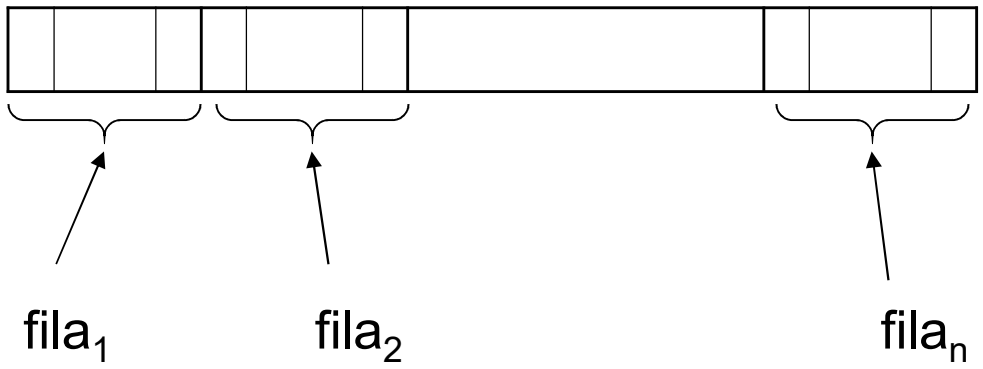
- El generador de código intermedio **solo** debería producir código para evaluar la fórmula *i*width+c*.

Arreglos Bidimensionales

- Pueden ser almacenados
  - Por **filas** or
  - Por **columnas**.

- La mayoría de los lenguajes de programación usan el método **por filas**.

- Esquema:



Arreglos Bidimensionales

- La ubicación de $A[i_1,i_2]$ es

$$base_A + ((i_1 - low_1)*n_2 + i_2 - low_2)*width$$

$base_A$ es la dirección de la primera posición del arreglo A.
low_1 es el límite inferior de la primera dimensión
low_2 es el límite inferior de la segunda dimensión
n_2 es el número de elementos de cada fila
width es el tamaño de cada elemento

- De nuevo, la fórmula puede ser re-escrita como

$$\underbrace{((i_1*n_2)+i_2)*width}_{\text{debería ser computado en ejecución}} + \underbrace{(base_A-((low_1*n_1)+low_2)*width)}_{\text{puede ser computado en compilación}}$$

Generación de Código Intermedio

- **Introducción.**
- **Lenguajes Intermedios.** Código de tres direcciones. Tipos de sentencias. Traducción a código intermedio. Posibles implementaciones.
- **Declaraciones.** Manejo de información de alcance.
- **Sentencias de Asignación.**
- **Expresiones Booleanas.**
- **Sentencias *Case.* Llamadas a Procedimiento**

Expresiones Booleanas

- Las expresiones booleanas son usadas para computar valores lógicos.
- A menudo son usadas en sentencias condicionales o repetitivas.
- Métodos para traducir expresiones booleanas:
  - Numéricos
    - True es representado como 1 y false como 0.
    - Valores positivos son considerados *true* y el cero es considerado *false*.
    - En forma análoga a las expresiones aritméticas.
  - Flujo de Control
    - El valor de un boolean es representado por una posición alcanzada en el programa
    - Usualmente, no es necesario evaluar completamente la expresión (*short circuit code*).

Expresiones Booleanas

Representación Numérica

- Las expresiones booleanas son tratadas como expresiones aritméticas, codificando los valores *true* y *false*.

- Ejemplo 1: *a or b and not c*

  t_1:= not c
  t_2:= b and t_1
  t_3:= a or t_2

- Ejemplo 2: *a < b*

```
100:    if a < b goto 103
101:    t:= 0
102:    goto 104
103:    t:= 1
104:    …
```

Expresiones Booleanas

Representación Numérica

Production	Semantic Rules
$E \rightarrow E_1$ or E_2	E.place := newtemp; emit(E.place ':=' E_1.place 'or' E_2.place)
$E \rightarrow E_1$ and E_2	E.place := newtemp; emit(E.place ':=' E_1.place 'and' E_2.place)
$E \rightarrow$ not E_1	E.place := newtemp; emit(E.place ':=' 'not' E_1.place)
$E \rightarrow (E_1)$	E.place := E1.place;

Expresiones Booleanas

Representación Numérica

Production	Semantic Rules

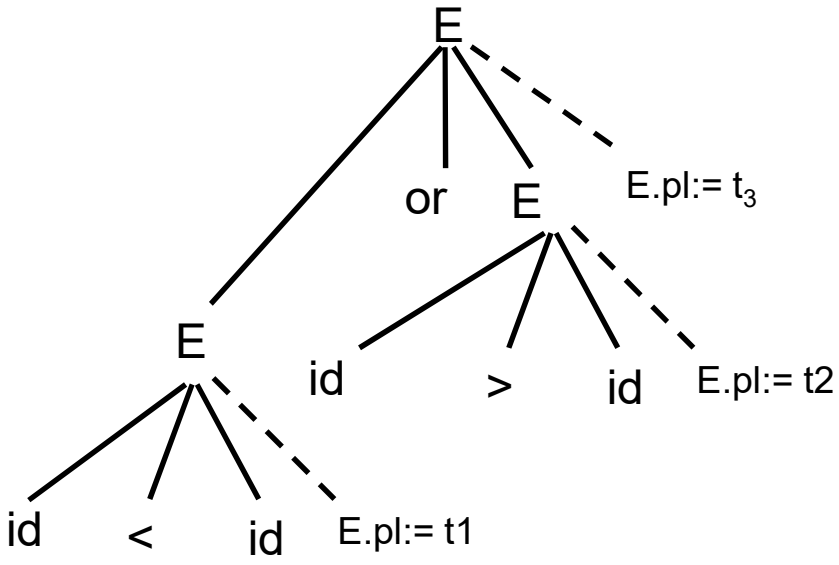
$E \rightarrow id_1$ relop id_2	E.place := newtemp;
	emit('if' id_1.place relop.op id_2.place 'goto'
	nextstat+3);
	emit(E.place ':=' '0');
	emit('goto' nextstat+2);
	emit(E.place ':=' '1');

E → true	E.place := newtemp; emit(E.place ':=' '1')
E → false	E.place := newtemp; emit(E.place ':=' '0')

Expresiones Booleanas

Representación Numérica. Ejemplo 1

a < b or c > d



Código Generado

```
100:    if a < b goto 103
101:    t1:= 0
102:    goto 104
103:    t1:= 1
104:    if c > d goto 107
105:    t2:= 0
106:    goto 108
107:    t2:= 1
108:    t3:= t1 or t2
```

Expresiones Booleanas

Representación Numérica. Ejemplo 2

a<b or c<d and e<f

```
100: if a < b goto 103
101: t1 := 0
102: goto 104
103: t1 := 1
104: if c < d goto 107
105: t2 := 0
106: goto 108
107: t2 := 1
108: if e < f goto 111
109: t3 := 0
110: goto 112
111: t3 := 1
112: t4 := t2 and t3
113: t5 := t1 or t4
```

Expresiones Booleanas

Representación por Flujo de Control

- Usamos una función *newlabel* que retorna una nueva etiqueta simbólica en cada llamado.

- Cada expresión booleana tiene dos nuevos atributos:

  - E.true: es el label destino si E es true
  - E.false: es el label destino si E es false

- Usamos un atributo *S.next* de una sentencia S:
  - Es un atributo heredado cuyo valor es el label asignado a la primera instrucción a ser ejecutada después del código para S.
  - Usado para evitar *saltos a saltos* (o *gotos a gotos*).

Expresiones Booleanas

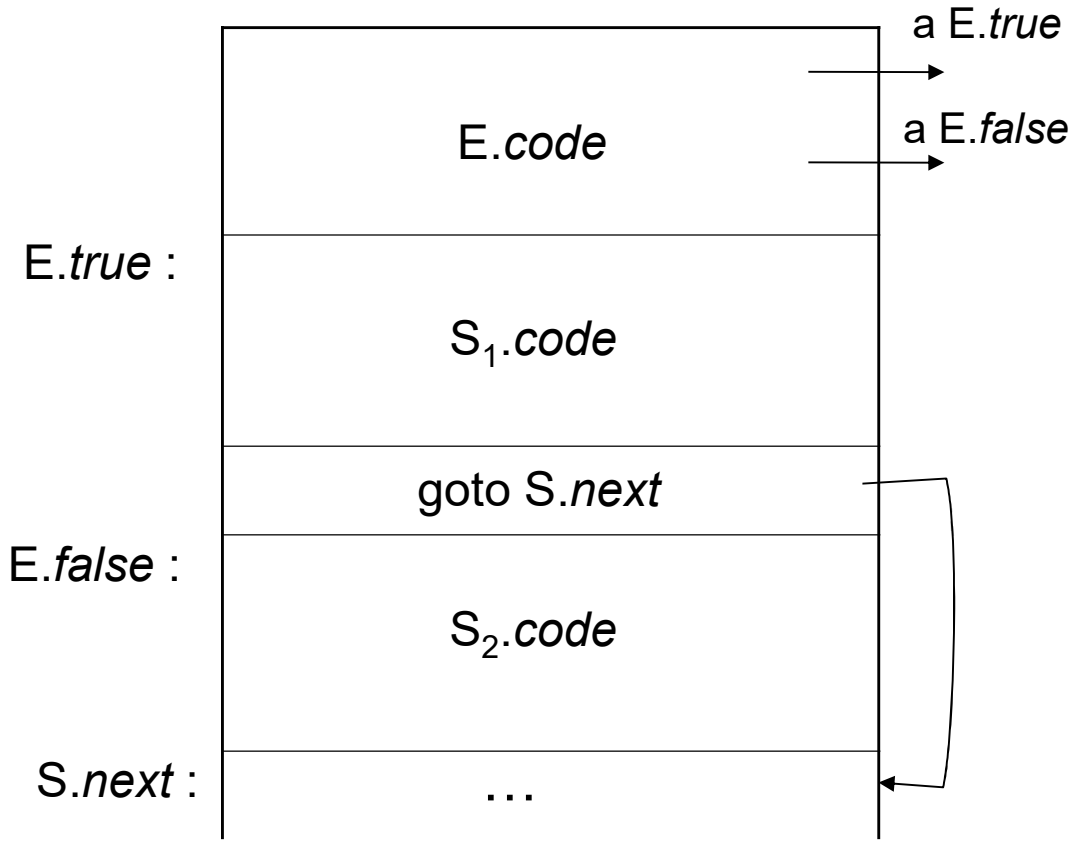
Representación por Flujo de Control – Condicionales

Production	Semantic Rules
$S \rightarrow$ if E then S_1	E.true := newlabel; E.false := S.next; S_1.next := S.next; S.code := E.code || gen(E.true ':') || S_1.code

Production	Semantic Rules
$S \rightarrow$ if E then S_1 else S_2	E.true := newlabel; E.false := newlabel; S_1.next := S.next; S_2.next := S.next; S.code := E.code || gen(E.true ':') || S_1.code || gen('goto' S.next) || gen(E.false ':') || S_2.code

Expresiones Booleanas

Representación por Flujo de Control – Caso if-then-else



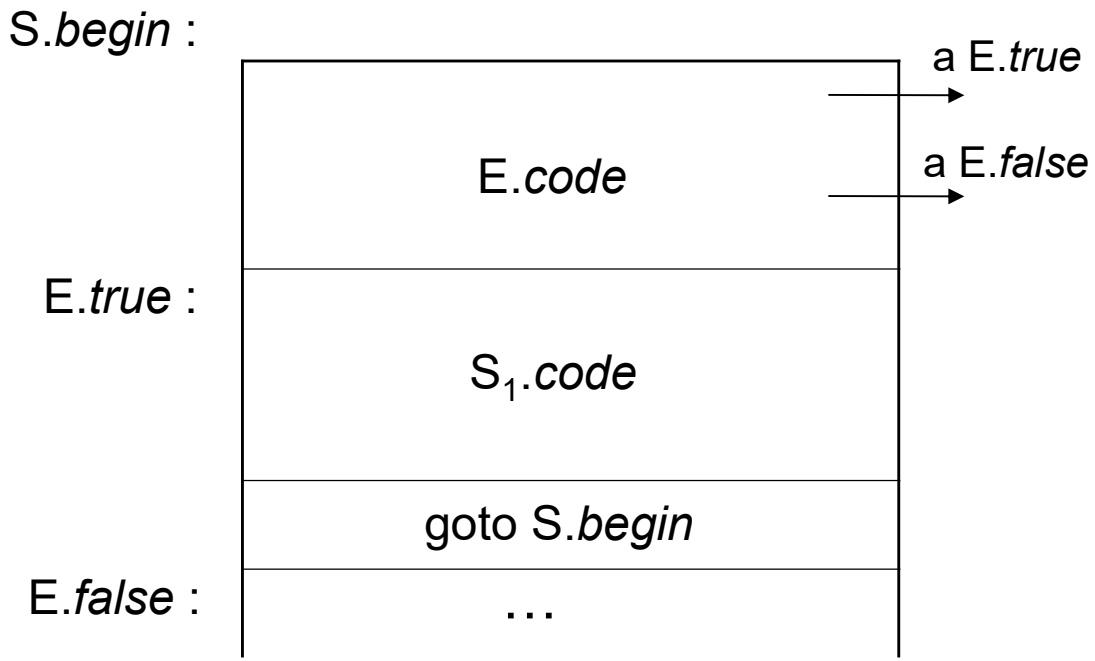
Expresiones Booleanas

Representación por Flujo de Control – Repetitivas

Production	Semantic Rules
$S \rightarrow$ while E do S_1	S.begin := newlabel; E.true := newlabel; E.false := S.next; S1.next := S.begin; S.code := gen(S.begin ':') || E.code || gen(E.true ':') || S_1.code || gen('goto' S.begin)

Expresiones Booleanas

Representación por Flujo de Control – Caso while-do



Expresiones Booleanas

Representación por Flujo de Control – Expresiones (re-visitadas)

Production	Semantic Rules
$E \rightarrow E_1$ or E_2	E_1.true := E.true; E_1.false := newlabel; E_2.true := E.true; E_2.false := E.false; E.code := E_1.code || gen(E_1.false ':') || E_2.code
$E \rightarrow E_1$ and E_2	E_1.true := newlabel; E_1.false := E.false; E_2.true := E.true; E_2.false := E.false; E.code := E_1.code || gen(E_1.true ':') || E_2.code

Expresiones Booleanas

Representación por Flujo de Control – Expresiones (re-visitadas)

Production	Semantic Rules
$E \rightarrow \text{not } E_1$	E_1.true := E.false; E_1.false := E.true; E.code := E_1.code
$E \rightarrow (E_1)$	E_1.true := E.true; E_1.false := E.false; E.code := E_1.code
$E \rightarrow id_1 \text{ relop } id_2$	E.code := gen('if' id.place relop.op id_2.place 'goto' E.true) || gen('goto' E.false)
E → true	E.code := gen('goto' E.true)
E → false	E.code := gen('goto' E.false)

Expresiones Booleanas

Representación por Flujo de Control

Ejemplo (re-visitado): *a<b or c<d and e<f*

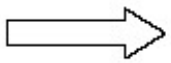
```
        if a < b goto Ltrue
        goto L1
L1:     if c < d goto L2
        goto Lfalse
L2:     if e < f goto Ltrue
        goto Lfalse
```

Expresiones Booleanas

Representación por Flujo de Control

Otro ejemplo:

```
while a < b do
  if c < d then
    x := y + z
  else
    x := y - z
```



```
L1:     if a < b goto L2
        goto Lnext
L2:     if c < d goto L3
        goto L4
L3:     t1 := y + z
        x:= t1
        goto L1
L4:     t2 := y – z
        x := t2
        goto L1
Lnext:
```

Expresiones Booleanas

Valores Booleanos y Flujo de Control

Como hemos visto, usamos expresiones booleanas para alterar el flujo de control en las sentencias.

Además, una expresión booleana también puede ser evaluada, por ejemplo en sentencias de asignación, como x:= a < b; .

Una manera *limpia* (pero costosa) de manejar ambos roles es construir primero un árbol sintáctico para las expresiones, usando alguno de los siguientes enfoques:

1. Hacer dos pasadas. Construir un árbol sintáctico para la entrada, luego hacer un recorrido *depth-first* y computar las traducciones especificadas por las reglas semánticas.

2. Hacer una pasada para sentencias, pero dos para expresiones. Aquí, *E* sería traducido en **while** (*E*) S1 antes que sea analizado S1.

La traducción de *E* sería realizada construyendo su árbol sintáctico y luego realizando un recorrido del árbol.

Expresiones Booleanas

Valores Booleanos y Flujo de Control

Consideremos la siguiente gramática:

S → **id** = E; | **if** (E) S | **while** (E) S | SS

E → E **or** E | E **and** E | E **rel** E | E + E | (E) | **id** | **true** | **false**

Podríamos manejar estos dos "*roles*" de *E*, empleando dos funciones de generación de código por separado.

Sup. que el atributo *E.n* denota el nodo del árbol sintáctico para la expresión *E* y supongamos que los nodos son objetos. Sea el método *jump*, que genera *jumping code* en un nodo y sea el método *value*, que genera código para computar el valor de un nodo en un determinado temporal.

Cuando *E* aparece en *S* → **while** (*E*) *S*, el método *jump* es llamado en el nodo *E.n*. La implementación de *jump* estaría basada en las reglas para traducir expresiones booleanas por flujo de control.

Expresiones Booleanas

Valores Booleanos y Flujo de Control

Consideremos la siguiente gramática:

S → **id** = E; | **if** (E) S | **while** (E) S | SS

E → E **or** E | E **and** E | E **rel** E | E + E | (E) | **id** | **true** | **false**

Cuando *E* aparece en *S* → **id** = *E*;, el método *value* es llamado en el nodo *E.n*.

Si *E* tiene la forma *E1* + *E2*, la llamada al método *E.n.value*() genera código de acuerdo a las reglas vistas en la representación numérica.

Si *E* tiene la forma *E1* **and** *E2*, primero se genera *jumping code* para *E* y luego se asigna *true* o false a un nuevo temporal *t* en los destinos *E.true* y *E.false*, respectivamente, del *jumping code*.

Expresiones Booleanas

Valores Booleanos y Flujo de Control

Por ejemplo, la asignación:

```
x:= a < b and c < d
```

Podría ser traducida al siguiente código:

```
     IfFalse a < b goto L1
     IfFalse c < d goto L1
     t:= true
     goto L2
L1:  t:= false
L2   x:= t
```

Generación de Código Intermedio

- **Introducción.**
- **Lenguajes Intermedios.** Código de tres direcciones. Tipos de sentencias. Traducción a código intermedio. Posibles implementaciones.
- **Declaraciones.** Manejo de información de alcance.
- **Sentencias de Asignación.**
- **Expresiones Booleanas.**
- **Sentencias *Case.* Llamadas a Procedimiento**

Sentencias Case

- Consideramos la sintaxis estilo C:

switch *E*
  begin
    case V_1: S_1
    case V_2: S_2
      …
    case V_{n-1}: S_{n-1}
    default: S_n
  end

Sentencias Case

- La traducción a código intermedio de una sentencia *case* consiste en código para:

  1. Evaluar la expresión
  2. Determinar qué valor de la lista de casos, se aplica de acuerdo al valor de la expresión
  3. Ejecutar la sentencia asociada el valor encontrado

- Opciones de Implementación del paso 2 (anterior):

  - Saltos condicionales (goto´s)
    - Menos de 10 casos
  - Tabla de pares (Valor,Label)
    - Más de 10 casos
    - Rango de valores *denso*
  - Tabla hash
    - Más de 10 casos
    - Rango de valores *dispersos*

Sentencias Case

Traducción a Código Intermedio. Primera aproximación

switch *E*
begin
case V_1: S_1
case V_2: S_2
…
case V_{n-1}: S_{n-1}
default: S_n
end

```
code to evaluate E into t
goto test
L1: code for S1
    goto next
    ...
Lk: code for Sk
    goto next
Ld: code for Sd
    goto next
test:
    if t = V1 goto L1
    ...
    if t = Vk goto Lk
    goto Ld
next:
```

Sentencias Case

Traducción a Código Intermedio. Segunda aproximación

switch *E*
begin
case V_1: S_1
case V_2: S_2
…
case V_{n-1}: S_{n-1}
default: S_n
end

```
code to evaluate E into t
    if t <> V1 goto L1
    code for S1
    goto next
L1: if t <> V2 goto L2
    code for S2
    goto next
    ...
L(k-1): if t <> Vk goto Lk
    code for Sk
    goto next
Lk: code for Sd
next:
```

Llamadas a Procedimiento

- Los procedimientos conforman una pieza fundamental de los lenguajes de programación.
- Es imperativo que el compilador genere código eficiente para llamadas y retornos de procedimientos o funciones.
- Gran parte del soporte para llamadas a procedimiento es provisto por el paquete de soporte en *run-time.*
- Gramática de ejemplo:

$S \rightarrow$ **call id** (*Elist*)
Elist $\rightarrow$ *Elist*, *E*
Elist $\rightarrow$ *E*

Secuencia de Llamada

- La secuencia de llamada puede variar en diferentes implementaciones del mismo lenguaje.

- Usualmente, ciertas acciones deben ser realizadas:

  - Se debe asignar espacio para el registro de activación del procedimiento llamado.
  - Los argumentos *pasados* deben ser evaluados y estar disponibles para el procedimiento llamado.
  - Los punteros de *entorno* deben posibilitar que el proc llamado acceda a los datos (locales y no locales) apropiados.
  - Debe almacenarse el estado del procedimiento llamador.
  - Debe almacenarse la dirección de retorno
  - Debe generarse una sentencia de transferencia de control al comienzo del código para el proc llamado.

Sentencias de Retorno

- Varias acciones deben realizarse al finalizar el procedimiento:
  - Si se trata de una función, el resultado debe ser almacenado.
  - Debe ser restaurado el registro de activación del procedimiento llamador.
  - Debe ser generada una sentencia de transferencia de control a la dirección de retorno del procedimiento llamador.
- No existe una división exacta de las tareas (en *run-time*) a cargo del procedimiento llamador y del llamado. Usualmente, esto depende de:
  - el lenguaje fuente
  - la máquina destino
  - el sistema operativo

Llamadas a Procedimiento

- Usualmente, el código para evaluar los parámetros es generado en primer lugar, seguido por las sentencias *param* y, finalmente, la sentencia *call*.

- Si no se quiere mezclar las sentencias de evaluación de argumentos con las sentencias *param*, se debería almacenar el valor de E.*place*, para cada expresión *E* en la llamada

  id(*E*, *E*, …, *E*)

- Si los parámetros son pasados (al proc. llamado) vía la *pila*, entonces las sentencias se entremezclan.

- Y, usualmente, la sentencia *param* es reemplazada en tiempo de generación de código por sentencias que apilan el parámetro en la pila.

Llamadas a Procedimiento

El siguiente esquema de traducción incorpora estas ideas:

S → **call id** (Elist) { **for** each item *p* on *queue* **do**
emit(‘param‘ *p*);
emit(‘call‘ **id**.*place*) }

Elist → Elist **,** E { append E.*place* to the end of *queue* }

Elist → E { initialize *queue* to contain only E.*place*}

Generación de Código Intermedio

- **Introducción.**
- **Lenguajes Intermedios.** Código de tres direcciones. Tipos de sentencias. Traducción a código intermedio. Posibles implementaciones.
- **Declaraciones.** Manejo de información de alcance.
- **Sentencias de Asignación.**
- **Expresiones Booleanas.**
- **Sentencias *Case.* Llamadas a Procedimiento**